## Exercise 1: Happy Supplies

**a. Assumptions:**

- Each part has a unique partNumber and partName.
- One part can be stored only in one cage.
- Each customer has a unique customerNumber.
- Different customers can have the same customerType.
- An employee can only process orders for one customer at a time.
- customerNumber, orderDate and orderTime together identify an order, which means a customer cannot place two orders at the same date and time.
- A cage can hold parts of different types, so cage code doesn't determine part type or price.
- Unit price is the part's standard price, the same on every order.

**b. Normalization Process**

The handwritten form is in UNF. One order has many partNumber, partName, partType, cageCode, quantityOrdered, and unitPrice, a repeating group made up of these attributes. Since customers usually provide their name when searching order details, we can keep customerName as one attribute for this database. Same for employees. There are values are non-atomic, for example, partNumber.

PK: **customerNumber, orderDate, orderTime** identify the order. This is NOT a true PK, because the part attributes repeat within one order. The repeating group is removed in 1NF.

| **customerNumber** | **orderDate** | **orderTime** | partNumber | partName | partType | cageCode | quantityOrdered | unitPrice | customerName | customerType | employeeName |
|---|---|---|---|---|---|---|---|---|---|---|---|
| HG54587 | 7/1/2024 | 10:30 am | 10654<br>10456<br>10776<br>10657 | Float Control Modulator Hose Assembly Float Assembly | Plumbing Electrical Plumbing Plumbing | G413<br>H433<br>G413<br>G413 | 4<br>3<br>7<br>5 | 12<br>7<br>9<br>10 | Jeff Peterson | Consumer | D. Harrison |

UNF → 1NF

Remove the repeating group by giving each partNumber its own row, repeating the order header on every row. All values are atomic.

The PK of 1NF is a composite key: **customerNumber**, **partNumber, orderDate, orderTime**.

| **customerNumber** | **partNumber** | **orderDate** | **orderTime** | partName | partType | cageCode | quantityOrdered | unitPrice | customerName | customerType | employeeName |
|---|---|---|---|---|---|---|---|---|---|---|---|
| HG54587 | 10654 | 7/1/2024 | 10:30 am | Float Control | Plumbing | G413 | 4 | 12 | Jeff Peterson | Consumer | D. Harrison |
| HG54587 | 10456 | 7/1/2024 | 10:30 am | Modulator | Electrical | H433 | 3 | 7 | Jeff Peterson | Consumer | D. Harrison |
| HG54587 | 10776 | 7/1/2024 | 10:30 am | Hose Assembly | Plumbing | G413 | 7 | 9 | Jeff Peterson | Consumer | D. Harrison |
| HG54587 | 10657 | 7/1/2024 | 10:30 am | Float Assembly | Plumbing | G413 | 5 | 10 | Jeff Peterson | Consumer | D. Harrison |

1NF → 2NF
Remove the partial dependencies: in 1NF, every non-key attribute except quantityOrdered depends on only part of the composite key, so each group needs to be moved to its own relation based on the part of the PK it depends on, with separate tables in the database.

- customerName and customerType depend only on customerNumber, so they move to a customer relation
- employeeName depends only on the (customerNumber, orderDate, orderTime), so it moves to a customer_order relation.
- partName, partType, cageCode, and unitPrice depend only on partNumber, so they move to a part relation.
- quantityOrdered depends on the full key (customerNumber, orderDate, orderTime, partNumber), so it stays in the order_detail relation.

2NF now has no partial dependencies, no repeating groups, and all values are atomic.

customer

| **customerNumber** | customerName | customerType |
|---|---|---|
| HG54587 | Jeff Peterson | Consumer |

customer_order

| **customerNumber** | **orderDate** | **orderTime** | employeeName |
|---|---|---|---|

| | | | |
|---|---|---|---|
| HG54587 | 7/1/2024 | 10:30 am | D. Harrison |

order_detail

| **customerNumber** | **orderDate** | **orderTime** | **partNumber** | quantityOrdered |
|---|---|---|---|---|
| HG54587 | 7/1/2024 | 10:30 am | 10654 | 4 |
| HG54587 | 7/1/2024 | 10:30 am | 10456 | 3 |
| HG54587 | 7/1/2024 | 10:30 am | 10776 | 7 |
| HG54587 | 7/1/2024 | 10:30 am | 10657 | 5 |

part

| **partNumber** | partName | partType | cageCode | unitPrice |
|---|---|---|---|---|
| 10654 | Float Control | Plumbing | G413 | 12 |
| 10456 | Modulator | Electrical | H433 | 7 |
| 10776 | Hose Assembly | Plumbing | G413 | 9 |
| 10657 | Float Assembly | Plumbing | G413 | 10 |

2NF → 3NF

Identify the transitive dependencies:

- customer: customerName does not determine customerType, or vice versa.
- customer_order: only one non-key attribute.
- part**:** cageCode does not determine partType or unitPrice, because a cage can hold different types of parts (assumption). partType doesn't determine anything else either. There is no transitive dependency.
- order_detail**:** only one non-key attribute, so a transitive dependency isn't possible.

There are no transitive dependencies exist in 2NF, so the tables above are already in 3NF.

customer

| **customerNumber (PK)** | customerName | customerType |
|---|---|---|
| HG54587 | Jeff Peterson | Consumer |

customer_order

| ***customerNumber (PK, FK)*** | **orderDate (PK)** | **orderTime (PK)** | employeeName |
|---|---|---|---|
| HG54587 | 7/1/2024 | 10:30 am | D. Harrison |

order_detail

| ***customerNumber (PK, FK)*** | ***orderDate (PK, FK)*** | ***orderTime (PK, FK)*** | ***partNumber (PK , FK)*** | quantityOrdered |
|---|---|---|---|---|
| HG54587 | 7/1/2024 | 10:30 am | 10654 | 4 |
| HG54587 | 7/1/2024 | 10:30 am | 10456 | 3 |
| HG54587 | 7/1/2024 | 10:30 am | 10776 | 7 |
| HG54587 | 7/1/2024 | 10:30 am | 10657 | 5 |

part

| **partNumber (PK)** | partName | partType | cageCode | unitPrice |
|---|---|---|---|---|
| 10654 | Float Control | Plumbing | G413 | 12 |
| 10456 | Modulator | Electrical | H433 | 7 |
| 10776 | Hose Assembly | Plumbing | G413 | 9 |
| 10657 | Float Assembly | Plumbing | G413 | 10 |

**c. Identifying PKs and FKs**

| Step | PK | Relation/table |
|---|---|---|
| UNF | **customerNumber, orderDate, orderTime** (technically not a PK since there are repeating groups) | N/A because single table |

| | | |
|---|---|---|
| 1NF | **customerNumber, partNumber, orderDate, orderTime** | N/A because single table |
| 2NF | **customerNumber** | customer |
| | **customerNumber, orderDate, orderTime** | customer_order |
| | **customerNumber, orderDate, orderTime, partNumber** | order_detail |
| | **partNumber** | part |
| 3NF | No transitive dependencies were found, so the relations and their PKs are unchanged. | Same as 2NF |

3NF relations with primary and foreign keys

- customer (**customerNumber**, customerName, customerType)
- customer_order (***customerNumber***, **orderDate**, **orderTime**, employeeName)
- order_detail (***customerNumber***, ***orderDate***, ***orderTime***, ***partNumber***, quantityOrdered)
- part (***partNumber***, partName, partType, cageCode, unitPrice)

Foreign keys:

- customer_order.***customerNumber*** references customer.customerNumber. It links each order to the customer who placed it, and it is also part of customer_order's PK.
- order_detail.(***customerNumber, orderDate, orderTime***) is a composite FK that references customer_order. It links each line to its order.
- order_detail.***partNumber*** references part.partNumber. It links each line to the part ordered.

**d. Naming 3NF relations**

- **customer** (**customerNumber**, customerName, customerType): stores each customer's details once.
- **customer_order** (***customerNumber***, **orderDate**, **orderTime**, employeeName): stores each order and the employee who handled it.
- **order_detail** (***customerNumber***, ***orderDate***, ***orderTime***, ***partNumber***, quantityOrdered): stores each part line on an order and the quantity ordered.
- **part** (**partNumber**, partName, partType, cageCode, unitPrice): stores each part's details, its cage location and its standard price.

## Exercise 2: Panacea Mental Health Corporation

**a. Assumptions**

- Each therapist has a unique staffNo.
- Each patient has a unique patNo, corresponding to their patName.
- Each branchNo uniquely identifies one branch.
- staffNo, apptDate, apptTime can uniquely identify an appointment.
- A therapist can see only one patient at a specific date and time.
- A therapist works at only one branch on a given day.
- A patient cannot have two appointments at the same date and time.

**b. Normalization Process**

The existing data is in UNF. There are no repeating groups. However, there is technically non-atomic value under the appointment attribute since date and time are combined together. Therapist and patient names are treated as single atomic values, since the business identifies people by staffNo and patNo and has no stated need to search or sort by first or last name separately.

PK: **staffNo, appointment**.

| **staffNo** | therapistName | patNo | patName | **appointment** | | branchNo |
|---|---|---|---|---|---|---|
| | | | | date | time | |
| S1011 | Fred Smith | P100 | Lily White | 9/12/2022 | 10:00 | M15 |
| S1011 | Fred Smith | P105 | Jill Baker | 9/12/2022 | 12:00 | M15 |
| S1024 | Heidi Pierce | P108 | Andy McKee | 9/12/2022 | 10:00 | Q10 |
| S1024 | Heidi Pierce | P108 | Andy McKee | 9/14/2022 | 14:00 | Q10 |
| S1032 | Richard Levin | P105 | Jill Baker | 9/14/2022 | 16:30 | M15 |
| S1032 | Richard Levin | P110 | Jimmy Winter | 9/15/2022 | 18:00 | B13 |

UNF → 1NF

To remove non-atomic values, we need to split appointment to apptDate and apptTime. There are no repeating groups and all values are atomic now.

The PK of 1NF is a composite key: **staffNo, apptDate, apptTime**

| **staffNo** | therapistName | patNo | patName | **apptDate** | **apptTime** | branchNo |
|---|---|---|---|---|---|---|
| S1011 | Fred Smith | P100 | Lily White | 9/12/2022 | 10:00 | M15 |

| | | | | | | |
|---|---|---|---|---|---|---|
| S1011 | Fred Smith | P105 | Jill Baker | 9/12/2022 | 12:00 | M15 |
| S1024 | Heidi Pierce | P108 | Andy McKee | 9/12/2022 | 10:00 | Q10 |
| S1024 | Heidi Pierce | P108 | Andy McKee | 9/14/2022 | 14:00 | Q10 |
| S1032 | Richard Levin | P105 | Jill Baker | 9/14/2022 | 16:30 | M15 |
| S1032 | Richard Levin | P110 | Jimmy Winter | 9/15/2022 | 18:00 | B13 |

1NF → 2NF
Remove the partial dependencies:

- therapistName depends only on staffNo, so it moves to a therapist relation with staffNo as its PK.
- branchNo depends only on staffNo and apptDate, because a therapist works at only one branch on a given day. It moves to therapist_branch with (staffNo, apptDate) as its PK.
- patNo depends on the full key, since a therapist sees one specific patient at a given date and time, so it stays unchanged in patient_appt.
- patName depends on patNo, which is not part of the PK, so it is not a partial dependency. It stays in patient_appt for now and is handled in 3NF as a transitive dependency.

All three relations are now in 2NF: every non-key attribute depends on the whole PK of its relation.

therapist

| **staffNo** | therapistName |
|---|---|
| S1011 | Fred Smith |
| S1024 | Heidi Pierce |
| S1032 | Richard Levin |

therapist_branch

| **staffNo** | **apptDate** | branchNo |
|---|---|---|
| S1011 | 9/12/2022 | M15 |
| S1024 | 9/12/2022 | Q10 |
| S1024 | 9/14/2022 | Q10 |
| S1032 | 9/14/2022 | M15 |
| S1032 | 9/15/2022 | B13 |

patient_appt

| **staffNo** | **apptDate** | **apptTime** | patNo | patName |
|---|---|---|---|---|
| S1011 | 9/12/2022 | 10:00 | P100 | Lily White |
| S1011 | 9/12/2022 | 12:00 | P105 | Jill Baker |
| S1024 | 9/12/2022 | 10:00 | P108 | Andy McKee |
| S1024 | 9/14/2022 | 14:00 | P108 | Andy McKee |
| S1032 | 9/14/2022 | 16:30 | P105 | Jill Baker |
| S1032 | 9/15/2022 | 18:00 | P110 | Jimmy Winter |

2NF → 3NF
Remove transitive dependencies based on 2NF:

- patName depends on patNo, which is not part of the PK, so it is a transitive dependency. It moves to a patient relation with only patNo as PK, and patNo remains in patient_appt as a foreign key.
- therapist and therapist_branch each have only one non-key attribute, so a transitive dependency isn't possible. They stay unchanged.

therapist

| **staffNo (PK)** | therapistName |
|---|---|
| S1011 | Fred Smith |
| S1024 | Heidi Pierce |
| S1032 | Richard Levin |

therapist_branch

| ***staffNo (PK, FK)*** | **apptDate (PK)** | branchNo |
|---|---|---|
| S1011 | 9/12/2022 | M15 |
| S1024 | 9/12/2022 | Q10 |
| S1024 | 9/14/2022 | Q10 |
| S1032 | 9/14/2022 | M15 |
| S1032 | 9/15/2022 | B13 |

patient

| **patNo (PK)** | patName |
|---|---|
| P100 | Lily White |
| P105 | Jill Baker |
| P108 | Andy McKee |
| P110 | Jimmy Winter |

patient_appt

| ***staffNo (PK, FK)*** | ***apptDate (PK, FK)*** | **apptTime (PK)** | *patNo (FK)* |
|---|---|---|---|
| S1011 | 9/12/2022 | 10:00 | P100 |
| S1011 | 9/12/2022 | 12:00 | P105 |
| S1024 | 9/12/2022 | 10:00 | P108 |
| S1024 | 9/14/2022 | 14:00 | P108 |
| S1032 | 9/14/2022 | 16:30 | P105 |
| S1032 | 9/15/2022 | 18:00 | P110 |

**c. Identifying PKs and FKs**

| Step | PK | Relation/table |
|---|---|---|
| UNF | **staffNo, appointment** | N/A because single table |
| 1NF | **staffNo, apptDate, apptTime** | N/A because single table |
| 2NF | **staffNo** | therapist |
| | **staffNo, apptDate** | therapist_branch |
| | **staffNo, apptDate, apptTime** | patient_appt |
| 3NF | **staffNo** | therapist |
| | ***staffNo*, apptDate** | therapist_branch |
| | ***staffNo*, apptDate, apptTime** | patient_appt |
| | **patNo** | patient |

3NF relations with primary and foreign keys

- therapist (**staffNo,** therapistName)
- therapist_branch (***staffNo*, apptDate**, branchNo)
- patient (**patNo**, patName)
- patient_appt (***staffNo*, *apptDate*, apptTime**, *patNo*)

Foreign keys

- therapist_branch.***staffNo*** references therapist.staffNo. It links each branch assignment to the therapist, and it is also part of therapist_branch's PK.
- patient_appt.(***staffNo*, *apptDate***) is a composite FK that references therapist_branch. It links each appointment to the branch where that therapist works on that day.
- patient_appt.*patNo* references patient.patNo. It links each appointment to the patient seen.

**d. Naming 3NF relations**

- therapist (**staffNo**, therapistName): stores each therapist's details once.
- therapist_branch (***staffNo*, apptDate**, branchNo): stores which branch each therapist works at on a given day.
- patient (**patNo**, patName): stores each patient's details once.
- patient_appt (***staffNo*, *apptDate*, apptTime**, *patNo*): stores each appointment, recording which therapist sees which patient at a given date and time.

## Exercise 3: The Maid Better temp agency

### a. Assumptions:

- eventNo is unique to each event.
- contractNo uniquely identifies a contract.
- Each contract applies to one event.
- Each event only takes place in one eventLoc.
- Each employee can work on multiple contracts, and each contract can involve multiple employees.
- Each employee appears only once per contract, and hours represents the total hours worked by that employee on that contract. We can use (eNo, contractNo) to uniquely identify each record.
- An event can have multiple contracts, but the event only takes place in one eventLoc.
- Employee names are treated as atomic values.

### b. Normalization Process

The existing data is in 1NF. All of its values are atomic, and there are no repeating groups.

1NF PK is a composite key: **eNo, contractNo**

| **eNo** | **contractNo** | hours | eName | eventNo | eventLoc |
|---|---|---|---|---|---|
| 1135 | C1024 | 16 | Smith J | H25 | Queens |
| 1057 | C1024 | 24 | Hocine D | H25 | Queens |
| 1068 | C1025 | 28 | White T | H4 | Yonkers |
| 1135 | C1025 | 15 | Smith J | H4 | Yonkers |
| 1135 | C1026 | 10 | Smith J | H25 | Queens |

1NF → 2NF
Remove the partial dependencies:

- eName depends only on eNo, so it moves to an employee relation with eNo as its PK.
- eventNo depends only on contractNo, because each contract applies to only one event. eventLoc depends on eventNo, so it also depends only on contractNo. Both move to a contract relation with contractNo as its PK.
- hours depends on the full key, since it records the hours one employee worked on one contract, so it stays in employee_contract.

employee

| **eNo** | eName |
|---|---|

| | |
|---|---|
| 1135 | Smith J |
| 1057 | Hocine D |
| 1068 | White T |

contract

| **contractNo** | eventNo | eventLoc |
|---|---|---|
| C1024 | H25 | Queens |
| C1025 | H4 | Yonkers |
| C1026 | H25 | Queens |

employee_contract

| **eNo** | **contractNo** | hours |
|---|---|---|
| 1135 | C1024 | 16 |
| 1057 | C1024 | 24 |
| 1068 | C1025 | 28 |
| 1135 | C1025 | 15 |
| 1135 | C1026 | 10 |

2NF → 3NF:
Remove the transitive dependencies:

- employee and employee_contract each have only one non-key attribute, so a transitive dependency isn't possible. They stay unchanged.
- eventLoc depends on eventNo since one event can only take place in one eventLoc, and eventNo depends on contractNo (contractNo → eventNo → eventLoc). One event can also have multiple contracts, so eventLoc moves to a new event relation with eventNo as its PK, and eventNo remains in contract as a foreign key.

employee

| **eNo (PK)** | eName |
|---|---|
| 1135 | Smith J |
| 1057 | Hocine D |

| | |
|---|---|
| 1068 | White T |

employee_contract

| ***eNo (PK, FK)*** | ***contractNo (PK, FK)*** | hours |
|---|---|---|
| 1135 | C1024 | 16 |
| 1057 | C1024 | 24 |
| 1068 | C1025 | 28 |
| 1135 | C1025 | 15 |
| 1135 | C1026 | 10 |

contract

| **contractNo (PK)** | *eventNo (FK)* |
|---|---|
| C1024 | H25 |
| C1025 | H4 |
| C1026 | H25 |

event

| **eventNo (PK)** | eventLoc |
|---|---|
| H25 | Queens |
| H4 | Yonkers |

**c. Identifying PKs and FKs**

| Step | PK | Relation/table |
|---|---|---|
| 1NF | **eNo, contractNo** | N/A because single table |
| 2NF | **eNo** | employee |
| | **contractNo** | contract |
| | **eNo, contractNo** | employee_contract |

| | | |
|---|---|---|
| 3NF | **<u>eNo</u>** | employee |
| | ***<u>eNo, contractNo</u>*** | employee_contract |
| | **<u>contractNo</u>** | contract |
| | **<u>eventNo</u>** | event |

3NF relations with primary and foreign keys

- employee (**<u>eNo</u>,** eName)
- employee_contract (***<u>eNo, contractNo</u>***, hours)
- contract (**<u>contractNo</u>**, *eventNo*)
- event (**<u>eventNo</u>**, eventLoc)

Foreign keys

- employee_contract.***<u>eNo</u>*** references employee.eNo. It links each record to the employee who worked the hours, and it is also part of employee_contract's PK.
- employee_contract.***contractNo*** references contract.contractNo. It links each record to the contract the hours were worked on, and it is also part of employee_contract's PK.
- contract.*eventNo* references event.eventNo. It links each contract to the event it applies to.

**d. Naming 3NF relations**

- employee (**<u>eNo</u>**, eName): stores each employee's details once.
- employee_contract (***<u>eNo, contractNo</u>***, hours): records the hours each employee worked on each contract.
- contract (**<u>contractNo</u>**, *eventNo*): stores each contract and the event it applies to.
- event (**<u>eventNo</u>**, eventLoc): stores each event and its location once.